Module 18 — Production Projects, Labs and Portfolio Blueprint

> **Goal:** Turn the course into verifiable portfolio work: a CPU-first lab environment with a mock LLM, and five projects (RAG with an eval gate, serving capacity report, durable agent, eval-gated release pipeline, multi-tenant gateway capstone), each with objective, architecture, time-boxed milestones, measurable acceptance criteria, starter code, stretch goals and interview talking points.

Table of Contents

- 18.1 Lab Environment and Repository Conventions (CPU-first)
- 18.2 Project A: Production RAG Service with an Eval Gate
- 18.3 Project B: Serving Benchmark and Capacity Report
- 18.4 Project C: Durable Agent with Human Approval, Idempotency and Budgets
- 18.5 Project D: Eval-Gated Release Pipeline (Judge Calibration, Canary, Rollback)
- 18.6 Capstone: Multi-Tenant Gateway Slice and Portfolio Packaging

18.1 Lab Environment and Repository Conventions (CPU-first)

Conceptual Deep Dive

Every lab should run on a laptop with a deterministic **mock** LLM, and optionally against a real endpoint by changing environment variables (`LLM_BASE_URL`, `LLM_MODEL`, `LLM_API_KEY`, `MAX_TOKENS_PER_RUN`). Reuse the services in the repository's `docker-compose.yml` (Qdrant, Redis, LangFuse, optional vLLM or Ollama) rather than inventing a stack per project. The mock must speak the OpenAI-compatible chat-completions schema including streaming, so the same client code runs against it and against a real provider; tests then exercise your code without calling a model.

Recommended layout: `src/`, `tests/`, `evals/` (golden sets and gate configs), `infra/` (compose, manifests), `docs/adr/` (architecture decision records), `RESULTS.md` (measured numbers with configs), a `Makefile` with `make test`, `make eval` and `make bench`, and pinned requirements. A minimal Makefile:

```
test:  ; python -m pytest -q
eval:  ; python evals/run.py --gate evals/gate.json
bench: ; python bench/run.py --out results/latest.json
```

Conventions: fix and print random seeds; every experiment writes a JSON result containing its config and the git SHA; never commit secrets (use .env.example and .gitignore); add pre-commit hooks and a CI skeleton from day one. The **definition of done** for any lab: tests pass, the README has an architecture diagram and a results table, failure drills are documented, and a "what is simulated" statement is present.

Every project in this chapter follows one format: objective, architecture, milestones with time boxes, measurable acceptance criteria, starter code, stretch goals and interview talking points. Time boxes are planning guides, typically a few evenings each; adjust to your pace but keep them, because open-ended projects rarely finish.

Edge cases. Without a GPU, label simulated results as simulation and never present them as measured hardware numbers; a small model on CPU through llama.cpp or Ollama is a legitimate measurement of that setup only. Docker networking differs across Windows, macOS and Linux (host.docker.internal, host networking availability), so document the compose profile that works on each and prefer service names inside the compose network.

Cross-Connection Scenarios

- The mock server stands in for providers behind the gateway in 17.3 and for models in every later project.
- Golden sets and gates connect to Module 07 and Module 13; tracing to Module 10.
- Cost guard variables echo budget enforcement in Module 09.
- The checklist linter in 18.6 audits these conventions.

Production Anti-Patterns

- Labs that only work with a paid API key, so nobody can reproduce them and tests become flaky and costly.
- Reporting results without the config and commit that produced them.
- Committing `.env` files, leaking keys into history.

Minimal Working Code

A minimal OpenAI-compatible fake server returns deterministic completions, with optional SSE streaming and usage fields.

```
import json, threading
from http.server import BaseHTTPRequestHandler, HTTPServer

class Mock(BaseHTTPRequestHandler):
    def log_message(self, *a): pass

    def do_POST(self):
        body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        last = body["messages"][-1]["content"]
        text = f"MOCK ANSWER: {last}"
        usage = {"prompt_tokens": len(last.split()), "completion_tokens": len(text.sp
lit())}
        self.send_response(200)
        if body.get("stream"):
            self.send_header("Content-Type", "text/event-stream"); self.end_headers()
            for w in text.split():
                chunk = {"choices": [{"delta": {"content": w + " "
;}}]}
                self.wfile.write(f"data: {json.dumps(chunk)}\n\n".encode())
            self.wfile.write(f"data: {json.dumps({'choices': [], 'usage': usage})}\n\ndata: [DONE]\n
\n".encode())
        else:
            out = {"choices": [{"message": {"role": "assistant",
"content": text}}], "usage": usage}
            self.send_header("Content-Type", "application/json"); self.end_headers()
            self.wfile.write(json.dumps(out).encode())

def start_server() -> tuple[HTTPServer, str]:
    srv = HTTPServer(("127.0.0.1", 0), Mock)          # port 0 = any free port
    threading.Thread(target=srv.serve_forever, daemon=True).start()
    return srv, f"http://127.0.0.1:{srv.server_port}"
```

A smoke-test client asserts the schema and streaming behaviour against the server above (run it as `make smoke`).

```
import json, urllib.request

def call(base: str, prompt: str, stream: bool = False):
    req = urllib.request.Request(base + "/v1/chat/completions", method="POST",
                                 data=json.dumps({"model": "MODEL_ID", "stream&q
uot;: stream,
                                                  "messages": [{"role": "user&qu
ot;, "content": prompt}]}).encode(),
                                 headers={"Content-Type": "application/json"})
    with urllib.request.urlopen(req, timeout=5) as r:
        return r.read().decode()

srv, base = start_server()
```

```
plain = json.loads(call(base, "hello world"))
assert plain["choices"][0]["message"]["content"] == "MOCK ANSWER: hell
o world"
assert plain["usage"]["prompt_tokens"] == 2
events = [l[6:] for l in call(base, "hello world", stream=True).splitlines() if l.startswith(&q
uot;data: ")]
assert events[-1] == "[DONE]" and len(events) >= 4
streamed = "".join(json.loads(e)["choices"][0]["delta"]["content"
] for e in events[:-2])
assert streamed.strip() == "MOCK ANSWER: hello world"
srv.shutdown()
print("smoke test passed:", len(events), "stream events")
```

Interview Angle

Q: How do you structure a portfolio project so an interviewer can verify your claims in five minutes? **A:** A README with architecture diagram, a results table linked to a config and commit, one command (make test eval) that reproduces numbers on CPU, and an explicit list of what is simulated. Trade-off: reproducibility costs setup effort and limits scale claims. Check: a fresh clone passes tests and prints the headline metric within five minutes.

Q: How do you test LLM code without calling an LLM? **A:** Inject the model client behind an interface, use a deterministic mock for logic and schema tests, record and replay real responses for regression, and keep a small live-model suite for periodic smoke runs. Trade-off: mocks can hide model-specific behaviour. Check: mocked tests run in seconds and a nightly live suite catches schema drift.

Q: What goes in RESULTS.md? **A:** For each experiment: hypothesis, config (including seeds, model id, data version), commit SHA, metric with interval, comparison to a baseline, and conclusion with caveats, labelled measured or simulated. Trade-off: honest caveats reduce impressive headlines but raise credibility. Check: every number in the README traces to a RESULTS.md row.

18.2 Project A: Production RAG Service with an Eval Gate

Conceptual Deep Dive

Objective. Ingest a small public corpus, serve answers with citations, enforce ACL filtering, support document deletion, and block quality regressions in CI.

Architecture. Ingestion (parse, parent-child chunking, embed) feeds hybrid retrieval (your own BM25 plus an embedding index or a deterministic stub, fused with reciprocal rank fusion), an optional reranker, prompt assembly with a token budget, generation, and output checks (citation present, grounded). Tracing wraps every step. Build a golden set of 50 or more questions with expected source documents; metrics are hit@k, MRR and a faithfulness measure (LLM judge or heuristic). Test retrieval separately from generation, because the generator is non-deterministic.

Milestones (time-boxed). 1: ingest plus BM25 (one evening). 2: dense retrieval plus RRF (one evening). 3: eval harness plus CI gate (one evening). 4: ACL filtering plus a deletion drill (one evening). 5: tracing plus latency budget (one evening).

Acceptance criteria (set and justify them yourself): recall@5 at or above a target you justify from the baseline; p95 retrieval latency within a stated budget; zero ACL leaks in a canary test where a restricted document is planted; the gate fails on an injected regression such as shrinking top-k or breaking tokenisation.

Failure drills. Delete a document and prove it never appears (index, cache, citations); change an ACL and prove retrieval changes immediately; swap the embedding model via an alias and re-evaluate. **Stretch:** a reranker, query rewriting (6.8), graph links (6.6).

Talking points. Why hybrid retrieval, how you measured, and what broke. Edge cases: tiny corpora make every method look good, so include distractors and near-duplicates (same topic, different facts); and because generators vary run to run, assert on retrieval metrics and on structured properties of answers rather than exact strings.

Cross-Connection Scenarios

- Chunking, hybrid search and RRF (Module 06, Module 07) are the core techniques; deletion semantics use 8.7.
- The gate follows 13.2; tracing follows Module 10.
- Tenant ACL tests reuse the canary approach in 17.2.

- Governance documentation (16.3) can be generated for this service.

Production Anti-Patterns

- Reporting only a handful of demo queries as evidence of quality.
- Writing the golden set after tuning, which makes the metrics circular.
- Skipping deletion and ACL tests, leaving the riskiest promises unverified.

Minimal Working Code

A BM25 index with tokenisation, add and remove, and tests.

```
import math, re
from collections import Counter

class BM25:
    def __init__(self, k1=1.5, b=0.75):
        self.k1, self.b, self.docs = k1, b, {}

    @staticmethod
    def tok(text: str) -> list[str]:
        return re.findall(r"[a-z0-9]+", text.lower())

    def add(self, doc_id: str, text: str): self.docs[doc_id] = Counter(self.tok(text))
    def remove(self, doc_id: str): self.docs.pop(doc_id, None)

    def search(self, query: str, k: int = 3) -> list[tuple[str, float]]:
        n = len(self.docs)
        avg = sum(sum(c.values()) for c in self.docs.values()) / max(n, 1)
        scores = {}
        for t in set(self.tok(query)):
            df = sum(1 for c in self.docs.values() if t in c)
            idf = math.log(1 + (n - df + 0.5) / (df + 0.5))
            for d, c in self.docs.items():
                if t in c:
                    dl = sum(c.values())
                    scores[d] = scores.get(d, 0) + idf * c[t] * (self.k1 + 1) / (c[t] + self.k1 * (1 - se
lf.b + self.b * dl / avg))
        return sorted(scores.items(), key=lambda x: -x[1])[:k]

idx = BM25()
idx.add("a", "refund policy for damaged items"); idx.add("b", "shippin
g times by region"); idx.add("c", "refund refund refund")
assert idx.search("damaged refund")[0][0] == "a"
idx.remove("a")
assert "a" not in [d for d, _ in idx.search("damaged refund")]
print("bm25 tests passed", idx.search("refund"))
```

An eval runner computes hit@k and MRR, and a gate fails when a metric drops beyond a margin against a stored baseline.

```
import json

def evaluate(search, golden: list[dict], k: int = 5) -> dict:
    hits = rr = 0.0
    for item in golden:
        ranked = [d for d, _ in search(item["q"], k)]
        if item["gold"] in ranked:
            hits += 1
            rr += 1 / (ranked.index(item["gold"]) + 1)
    n = len(golden)
    return {"hit@k": round(hits / n, 3), "mrr": round(rr / n, 3)}

def gate(current: dict, baseline: dict, margin: float = 0.03) -> int:
    """Returns a process exit code: 0 pass, 1 fail."""
```

```
    bad = {m: (baseline[m], current[m]) for m in baseline if current[m] < baseline[m] - margin}
    print(json.dumps({"current": current, "baseline": baseline, "regressions&quo
t;: bad}))
    return 1 if bad else 0

golden = [{"q": "damaged refund", "gold": "a"}, {"q": &
quot;shipping region", "gold": "b"}]
good = lambda q, k: [("a", 1.0), ("b", 0.5)] if "refund" in q else [("
b", 1.0), ("a", 0.1)]
broken = lambda q, k: [("c", 1.0), ("a", 0.4)]                # injected regressi
on
baseline = evaluate(good, golden)
print("exit codes:", gate(evaluate(good, golden), baseline), gate(evaluate(broken, golden), bas
eline))
# in CI: sys.exit(gate(...)) makes the pipeline step fail
```

Interview Angle

Q: Walk me through your RAG project: what was the hardest failure and how did you detect it? **A:** Describe one concrete failure, for example a chunking change that dropped recall for table questions, caught by the golden-set gate rather than by users. Trade-off: gating slows merges and golden sets can overfit. Check: recall@5 before and after with intervals, plus the regression test added for that failure.

Q: How did you choose chunk size and prove it? **A:** Swept sizes and overlaps on the golden set, comparing hit@k and answer faithfulness, including a per-question-type slice, and picked the knee in quality versus index size and latency. Trade-off: larger chunks help recall of context but dilute scores. Check: paired comparison on the same questions with a bootstrap interval.

Q: How do you guarantee deleted documents disappear? **A:** Delete by stable id from the index, caches and derived stores, propagate through a tombstone, then verify with a probe query for unique content; backups expire on a schedule. Trade-off: eventual deletion windows exist; state them. Check: an automated drill that deletes a planted document and asserts it is absent from retrieval, cache and citations.

18.3 Project B: Serving Benchmark and Capacity Report

Conceptual Deep Dive

Objective. Produce a capacity report for a chosen model and engine on the hardware you have, or on a clearly labelled simulator: a load sweep, the knee, goodput under an SLO, cost per million tokens, a recommendation and its risks.

Method (from 5.8): open-loop arrivals (requests arrive on a schedule regardless of completions, as real users do), realistic input and output length distributions, warm-up excluded, repeated runs with bootstrap confidence intervals, and full disclosure of engine flags, max length and batch limits.

Milestones. 1: workload definition (length distributions, arrival process). 2: load generator. 3: sweep runs across offered rates. 4: analysis (percentiles, goodput, knee). 5: capacity plan (replicas for a target RPS with headroom and redundancy). 6: write-up in RESULTS.md.

Acceptance criteria. The report reproduces from a single command; includes TTFT and inter-token latency percentiles; a goodput curve with the knee identified; a cost model whose prices are placeholders flagged as perishable; and sensitivity to prompt length (for example short versus long prompts).

Without a GPU: run against the mock server with a queueing model and say so, or measure a small model on CPU with Ollama or llama.cpp and label the hardware. **Stretch:** prefix-cache on versus off (5.6), quantisation comparison (4.7), cache-aware routing simulation.

Talking points. What the knee means (the load beyond which latency grows faster than throughput), and why averages lie (tails dominate user experience).

Edge cases. The client can be the bottleneck (single-threaded load generator, laptop thermal throttling): verify generator CPU and rate accuracy before blaming the server. Engines default to different max-length and batch settings, so comparisons must align them or disclose the differences.

Cross-Connection Scenarios

- Method and metrics: Module 05 (5.6, 5.8); quantisation variants: Module 04 (4.7).
- Per-replica goodput feeds autoscaling targets and warm capacity in 17.4.
- Cost model formulas: Module 11 and Module 12 section 2.1.
- Alerts on the knee relate to SLO burn in Module 10.

Production Anti-Patterns

- Quoting a single throughput number with no latency, length distribution or config.
- Closed-loop load only (fixed concurrency), which hides queueing collapse at a given arrival rate.
- No cost model, so the benchmark cannot drive a decision.

Minimal Working Code

A report generator turns raw per-request rows into percentiles, goodput under an SLO and a knee estimate.

```
import csv, io, random

def percentile(xs, p):
    xs = sorted(xs); return xs[min(len(xs) - 1, int(p / 100 * len(xs)))]

def report(rows: list[dict], slo_ms: float, duration_s: float) -> list[dict]:
    levels = sorted({float(r["offered_rps"]) for r in rows})
    out = []
    for lv in levels:
        ttft = [float(r["ttft_ms"]) for r in rows if float(r["offered_rps"]) == lv]
        ok = sum(t <= slo_ms for t in ttft)
        out.append({"rps": lv, "p50": percentile(ttft, 50), "p95": percenti
le(ttft, 95), "goodput": ok / duration_s})
    return out

def knee(table: list[dict]) -> float:
    base = (table[1]["goodput"] - table[0]["goodput"]) / (table[1]["rps"] -
 table[0]["rps"])
    for a, b in zip(table, table[1:]):
        if (b["goodput"] - a["goodput"]) / (b["rps"] - a["rps"])
< 0.5 * base:
            return a["rps"]
    return table[-1]["rps"]

rng, buf = random.Random(0), io.StringIO()      # simulated data: latency explodes near capacity 8 rps
w = csv.writer(buf); w.writerow(["offered_rps", "ttft_ms"])
for lv in (2, 4, 6, 8, 10, 12):
    for _ in range(int(lv * 30)):
        w.writerow([lv, 80 / max(0.05, 1 - min(lv, 9.5) / 10) * rng.uniform(0.8, 1.2)])
rows = list(csv.DictReader(io.StringIO(buf.getvalue())))
tab = report(rows, slo_ms=400, duration_s=30)
for r in tab: print(r)
print("knee at offered rps:", knee(tab))
```

A capacity-plan function converts measured goodput per replica into replica count and monthly cost with a placeholder price.

```
import math

def capacity_plan(target_rps: float, goodput_per_replica: float, headroom: float = 0.3,
                  redundancy: int = 1, price_per_gpu_hour: float = 2.0, gpus_per_replica: int = 1) ->
dict:
    """price_per_gpu_hour is a PLACEHOLDER (perishable) - replace with current quotes.&quo
t;"""
    needed = math.ceil(target_rps * (1 + headroom) / goodput_per_replica)
    replicas = needed + redundancy                      # N+k so one failure does not breach the SLO
    monthly = replicas * gpus_per_replica * price_per_gpu_hour * 730
    return {"replicas": replicas, "monthly_cost": round(monthly), "utilisation_a
t_target": round(target_rps / (replicas * goodput_per_replica), 2)}
```

```
print(capacity_plan(target_rps=40, goodput_per_replica=7.5))
print(capacity_plan(target_rps=40, goodput_per_replica=7.5, headroom=0.5, redundancy=2))
assert capacity_plan(40, 7.5)["replicas"] == 8
```

Interview Angle

Q: What does your benchmark show about where the system falls over, and how did you find it? **A:** Describe the sweep: goodput rises with offered load until the knee, after which TTFT percentiles explode (queueing) and goodput falls; identify the limiter (KV cache, batch limit, client). Trade-off: pushing near the knee raises utilisation and tail risk. Check: repeated runs with intervals and a rerun after the suspected fix moving the knee.

Q: How do you translate a benchmark into a fleet size? **A:** Take goodput per replica at your SLO, divide the target peak RPS plus headroom, add redundancy for failures and rollouts, and check cost; validate with a load test of the planned fleet. Trade-off: headroom costs money, too little breaks the SLO in bursts. Check: observed p95 at peak stays within SLO and utilisation within the planned band.

Q: Why did you choose goodput over raw throughput? **A:** Raw throughput counts requests that arrived too late to be useful; goodput counts requests meeting the SLO, which is what users and the budget care about. Trade-off: it depends on an SLO choice, so show sensitivity. Check: plot both curves; the gap between them marks overload.

18.4 Project C: Durable Agent with Human Approval, Idempotency and Budgets

Conceptual Deep Dive

Objective. Build a support-operations agent with mock tools (look up an order, propose a refund, request approval, execute the refund) that survives crashes, requires human approval, applies side effects exactly once and respects budgets.

Architecture. A state machine with checkpointing (LangGraph or hand-rolled), an outbox and idempotency table in SQLite or Postgres, an approval queue with expiry, a policy engine that decides which tool calls are allowed (9.6), a budget governor limiting steps, tokens and money (9.8), and tracing for every transition. Separate decision from execution: the model proposes, deterministic code authorises and executes.

Milestones. 1: happy path. 2: checkpoint and resume. 3: approvals with expiry. 4: idempotent side effects. 5: chaos test (kill the process at random points). 6: trajectory eval suite with pass^k (9.7).

Acceptance criteria. After 100 randomised crash runs there is exactly one refund effect per order (the practical meaning of **exactly-once** here is at-least-once execution plus an idempotent receiver keyed by a stable key); over-budget runs stop gracefully with a clear status; unauthorised tool calls are denied and logged.

Stretch. An MCP tool server with pinned tool definitions, and an injection test via a poisoned order note that tries to trigger an unapproved refund.

Talking points. Why decide/execute separation matters, and how you proved exactly-once (the chaos harness, not an argument).

Edge cases. A crash between the tool execution and the checkpoint write means the step reruns after resume; the idempotency key (derived from run id and step, passed to the downstream system) turns the repeat into a no-op. An approver may answer after the request expired: treat the response as stale, require fresh approval, and log it.

Cross-Connection Scenarios

- Durable state, checkpoints and HITL come from Module 09; policy and budgets from 9.6 and 9.8.
- Audit records for approvals follow 16.4; oversight quality follows 16.5.
- The chaos approach is the agent analogue of game days in 17.5.
- Tool pinning and injection tests link to Module 08.

Production Anti-Patterns

- Retry loops without idempotency keys, which duplicate refunds exactly when systems are unstable.
- Approval implemented as a prompt instruction ("ask the user first") rather than enforced state in code.
- Testing only the happy path and trusting that crashes behave.

Minimal Working Code

A tool executor with idempotency keys and an outbox table returns the stored result on duplicate execution.

```
import json, sqlite3

class Outbox:
    def __init__(self):
        self.db = sqlite3.connect(":memory:")
        self.db.execute("CREATE TABLE outbox(key TEXT PRIMARY KEY, result TEXT)")

    def execute_once(self, key: str, fn, *args):
        row = self.db.execute("SELECT result FROM outbox WHERE key=?", (key,)).fetchone()
        if row:
            return json.loads(row[0])                # duplicate: return stored result, no side effect
        result = fn(key, *args)                      # downstream also dedupes on the key
        self.db.execute("INSERT INTO outbox VALUES (?, ?)", (key, json.dumps(result)))
        self.db.commit()
        return result

class PaymentProvider:                               # fake downstream with its own idempotency
    def __init__(self): self.refunds = {}
    def refund(self, key: str, order: str, amount: int) -> dict:
        return self.refunds.setdefault(key, {"order": order, "amount": amount, "
id": f"rf-{len(self.refunds) + 1}"})

pay, box = PaymentProvider(), Outbox()
r1 = box.execute_once("run1:refund:o-17", pay.refund, "o-17", 40)
r2 = box.execute_once("run1:refund:o-17", pay.refund, "o-17", 40)
assert r1 == r2 and len(pay.refunds) == 1
print("duplicate suppressed:", r2)
```

A chaos harness crashes the workflow at random steps, resumes it, and asserts exactly-once effects over many seeds.

```
import random

class Crash(Exception): pass

def run_workflow(state: dict, box: Outbox, pay: PaymentProvider, rng: random.Random, crash_p: float):
    steps = ["lookup", "propose", "approve", "refund", "noti
fy"]
    for step in steps[state["done"]:]:
        if rng.random() < crash_p: raise Crash(f"before {step}")
        if step == "refund":
            box.execute_once(f"{state['run']}:refund", pay.refund, "o-17", 40)
            if rng.random() < crash_p: raise Crash("after refund, before checkpoint")   # th
e nasty window
        state["done"] += 1                           # durable checkpoint (simulated)

def chaos(seeds=100) -> int:
    bad = 0
    for s in range(seeds):
        rng, box, pay, state = random.Random(s), Outbox(), PaymentProvider(), {"run": f"r{
s}", "done": 0}
        while True:
            try:
                run_workflow(state, box, pay, rng, crash_p=0.25); break
            except Crash:
                continue                             # process restarted; resume from checkpoint
        bad += len(pay.refunds) != 1
    return bad

print("runs with wrong refund count:", chaos())
assert chaos() == 0
```

Interview Angle

Q: How did you prove your agent never double-refunds? **A:** Two layers: idempotency keys derived from run id and step enforced in an outbox and by the downstream, and a chaos harness crashing at random points including between side effect and checkpoint, asserting one refund per order across 100 or more seeds. Trade-off: exactly-once is really at-least-once plus idempotency. Check: zero duplicates across seeds, and a test showing duplicates when the key is removed.

Q: How do approvals work across process restarts? **A:** Approvals are persisted state with an id, payload hash, expiry and approver; the workflow pauses at a waiting state, and resume re-reads it. A changed payload or expired approval invalidates it. Trade-off: waiting runs hold state and need cleanup. Check: kill the process mid-wait, restart, approve, and confirm single execution plus an audit record.

Q: What is your agent's pass^k and how did you improve it? **A:** Pass^k is the probability that all k independent runs of a task succeed, which exposes inconsistency that pass@k hides. Measure it on the trajectory suite, inspect failing runs, then fix with tool-description clarity, stricter schemas and deterministic guards. Trade-off: more runs cost money. Check: pass^k rises on a held-out task set, not only the tuned one.

18.5 Project D: Eval-Gated Release Pipeline (Judge Calibration, Canary, Rollback)

Conceptual Deep Dive

Objective. A release pipeline for an LLM application in which a change to a prompt, model alias or index must pass lint, unit tests, replay tests, an eval gate, bundle packaging, canary analysis and promotion, with a timed **rollback drill**.

Architecture. A bundle manifest (13.1) pins model, prompt, index and config; a CI workflow file (for example a GitHub Actions YAML with jobs for lint, test, replay and gate, plus a manual promotion job) runs the stages; a gate script (13.2) compares candidate to baseline with a paired bootstrap; the LLM judge is calibrated against 100 or more human-labelled items (14.2) and agreement is reported; a canary splitter and analyser (13.3) compare the candidate to control on quality and latency; an alias registry (13.6) records which bundle is live.

Milestones. 1: manifest and registry. 2: CI with a replay cache so tests do not call models. 3: gate with paired bootstrap and noise-floor measurement. 4: canary simulation with an injected regression. 5: rollback drill under five minutes, documented in a runbook.

Acceptance criteria. A pull request with an injected regression is blocked; a canary with an injected latency regression rolls back automatically; every release has an evidence pack (eval report, judge calibration, canary result) linked from the registry. **Optional:** make the changed artefact a QLoRA adapter (15.6) or a quantised model (4.7).

Talking points. What the gate measures, the noise floor (run the gate twice on identical input and record variation), and the false-block rate (how often a no-op change is blocked).

Edge cases. Gate flakiness is a measurable property: report repeated-run variance and widen margins or increase sample size accordingly. Judge cost matters: cache judgments by input hash, sample for large sets, and use a cheaper judge for screening with a stronger one for borderline cases, re-checking calibration whenever the judge changes.

Cross-Connection Scenarios

- Manifests, gates, canaries and aliases: Module 13 (13.1, 13.2, 13.3, 13.6); judge calibration: 14.2.
- Governance gates (16.1) map onto pipeline stages; the gateway alias (17.3) is the traffic switch.
- Incident response (17.6) consumes the same release markers.
- The portfolio checklist (18.6) can verify the pipeline runs locally.

Production Anti-Patterns

- A pipeline that cannot be run locally, so developers bypass it and debugging is slow.
- Gate thresholds chosen after seeing results, which makes the gate decorative.
- A rollback that has never been rehearsed, so the first real one is improvised.

Minimal Working Code

A pipeline runner executes named stages in order, times them, handles exit codes and writes a JSON run report.

```
import json, subprocess, sys, time

def run_pipeline(stages: list[tuple[str, list[str]]]) -> dict:
    report = {"stages": [], "status": "passed"}
    for name, argv in stages:
        t0 = time.perf_counter()
        p = subprocess.run(argv, capture_output=True, text=True)
        report["stages"].append({"name": name, "exit": p.returncode, "
seconds": round(time.perf_counter() - t0, 3)})
        if p.returncode != 0:
            report["status"] = f"failed at {name}"
            break                                   # later stages are skipped, as in CI
    return report

py = lambda code: [sys.executable, "-c", code]
stages = [("lint", py("print('ok')")), ("unit", py("assert 1 + 1 == 2&
quot;)),
          ("eval_gate", py("import sys; sys.exit(1)")), ("package", py(&quo
t;print('never runs')"))]
rep = run_pipeline(stages)
print(json.dumps(rep))
assert rep["status"] == "failed at eval_gate" and len(rep["stages"]) == 3
```

A rollback-drill script promotes a bad bundle through an alias registry, detects the breach with an analyser stub and rolls back, reporting elapsed simulated time.

```
class AliasRegistry:
    def __init__(self): self.live, self.history = {"support-chat": "bundle-41"}, []
    def promote(self, alias: str, bundle: str):
        self.history.append((alias, self.live[alias])); self.live[alias] = bundle
    def rollback(self, alias: str):
        a, prev = next(h for h in reversed(self.history) if h[0] == alias); self.live[alias] = prev

def analyser(bundle: str, minute: int) -> bool:
    """Stub: bundle-42 has a latency regression visible from minute 3."""
    return bundle == "bundle-42" and minute >= 3

def drill(reg: AliasRegistry, check_every_min: int = 1, action_min: float = 0.5) -> float:
    reg.promote("support-chat", "bundle-42")
    minute = 0
    while not analyser(reg.live["support-chat"], minute):
        minute += check_every_min
    reg.rollback("support-chat")
    return minute + action_min

reg = AliasRegistry()
elapsed = drill(reg)
print(f"rollback drill: live={reg.live['support-chat']} elapsed={elapsed} min")
assert reg.live["support-chat"] == "bundle-41" and elapsed < 5
```

Interview Angle

Q: Describe your release pipeline and what it caught. **A:** Stages: lint, unit, replay, eval gate, package, canary, promote; the gate caught a prompt edit that raised helpfulness scores but dropped citation accuracy on one slice. Trade-off: stricter gates slow shipping and cost judge calls. Check: the gate blocked an injected regression and passed a no-op change in repeated runs.

Q: How do you know your gate is neither too loose nor too tight? **A:** Measure both error types: inject known regressions of different sizes (detection rate) and rerun no-op changes (false-block rate), then set margins from the noise floor. Trade-off: sensitivity versus false alarms; choose thresholds before looking at candidate results. Check: detection of a regression at the size that matters and a false-block rate below an agreed small figure.

Q: How fast can you roll back and how do you know? **A:** Alias flip to the previous bundle, with timed drills recorded in the runbook; fast because bundles are immutable and clients call aliases. Trade-off: rollback of a data or index change may be slower than of a

prompt. Check: drill duration measured under the target, repeated after pipeline changes.

18.6 Capstone: Multi-Tenant Gateway Slice and Portfolio Packaging

Conceptual Deep Dive

Objective. Integrate pieces into one demonstrable slice: a gateway with virtual keys and per-tenant token buckets, tenant-namespaced caching, a router with model aliases, tracing, cost attribution, and a cross-tenant leak test. This **capstone** shows platform thinking rather than one more chatbot.

Deliverables. An architecture diagram; ADRs for three key decisions (12.7); a RESULTS.md with measured numbers and configs; a runbook (rollback, kill switch, key rotation); a five-minute demo script; and a threat model (8.6) listing tenant-isolation abuse cases and controls.

Packaging for interviews. Prepare a two-minute pitch, a ten-minute deep-dive outline, three numbers you can defend (each traced to RESULTS.md), and three failure stories (what broke, how you detected it, what you changed). This is what turns a repository into a **portfolio** that supports your claims.

Repo hygiene. README sections (problem, architecture, quick start, results, limitations, what is simulated), tests passing in CI with a badge, a LICENSE file, no secrets, pinned dependencies, and a Docker path so the project runs on an interviewer's machine without setup surprises.

What not to claim: unmeasured numbers, production use you did not have, benchmarks without configs, and scale beyond what you tested. A modest, verified claim beats an impressive vague one. Finish with self-review against the rubric in 19.3 and fix the weakest area.

Edge cases. If an interviewer asks you to extend the project live, modularity matters: keep policies, routing and storage behind small interfaces so adding a rate-limit rule or a provider is a ten-minute change. When dependencies are missing on their machine, a one-command Docker or devcontainer path rescues the demo.

Cross-Connection Scenarios

- Gateway design and keys: 17.3; isolation and noisy neighbours: 17.2; leak testing and canaries: 17.2.
- Cost attribution: 11.8; tracing: Module 10; ADR and design-doc format: 12.7 and Module 12 section 3.5.
- Threat model: 8.6; governance documents: 16.3.
- Self-review rubric: 19.3.

Production Anti-Patterns

- Over-claiming scale ("handles millions of requests") from a laptop demo.
- No tests for the multi-tenant isolation promise, the one claim an interviewer will probe.
- A README that is only screenshots, with no instructions or limitations.

Minimal Working Code

A mini gateway core with per-tenant token buckets, tenant-namespaced cache keys, counters and isolation tests.

```
class Gateway:
    def __init__(self, rate_per_s=2.0, burst=4, now=lambda: 0.0):
        self.rate, self.burst, self.now = rate_per_s, burst, now
        self.buckets, self.cache, self.stats = {}, {}, {"requests": 0, "tokens": 0, &
quot;cache_hits": 0}

    def _allow(self, tenant: str) -> bool:
        tokens, last = self.buckets.get(tenant, (self.burst, self.now()))
        tokens = min(self.burst, tokens + (self.now() - last) * self.rate)
        ok = tokens >= 1
        self.buckets[tenant] = (tokens - 1 if ok else tokens, self.now())
        return ok

    def complete(self, tenant: str, prompt: str, model=lambda p: f"answer({p})") -> tuple[in
t, str]:
```

```
        if not self._allow(tenant):
            return 429, "rate limited"
        self.stats["requests"] += 1
        key = f"{tenant}::{prompt}"                    # tenant-namespaced cache key
        if key in self.cache:
            self.stats["cache_hits"] += 1
            return 200, self.cache[key]
        out = self.cache[key] = model(prompt)
        self.stats["tokens"] += len(out.split())
        return 200, out

clock = [0.0]
gw = Gateway(now=lambda: clock[0])
assert gw.complete("acme", "secret-acme-plan")[0] == 200
gw.cache["acme::private"] = "ACME-ONLY"
assert gw.complete("globex", "private")[1] != "ACME-ONLY"          # no c
ross-tenant cache hit
codes = [gw.complete("globex", f"q{i}")[0] for i in range(6)]
assert codes.count(429) >= 2 and gw.complete("acme", "x")[0] == 200  # globex burs
t did not throttle acme
print(gw.stats, codes)
```

A portfolio checklist linter scans a directory for README sections, tests, RESULTS.md, LICENSE and secret-like patterns.

```
import re, tempfile
from pathlib import Path

SECRET = re.compile(r"(api[_-]?key|secret|token)\s*[=:]\s*['\"]?[A-Za-z0-9_\-]{16,}", re.I
)
SECTIONS = ["architecture", "quick start", "results", "limitations&quo
t;, "simulated"]

def lint(root: Path) -> dict[str, bool]:
    readme = (root / "README.md").read_text(encoding="utf-8").lower() if (root / &quo
t;README.md").exists() else ""
    checks = {f"README has '{s}'": s in readme for s in SECTIONS}
    checks["tests present"] = any(root.glob("tests/test_*.py"))
    checks["RESULTS.md present"] = (root / "RESULTS.md").exists()
    checks["LICENSE present"] = (root / "LICENSE").exists()
    leaked = [p.name for p in root.rglob("*") if p.is_file() and p.suffix in {".py",
".env", ".md", ".json"}
              and SECRET.search(p.read_text(encoding="utf-8", errors="ignore"))]
    checks["no secret-like strings"] = not leaked
    return checks

with tempfile.TemporaryDirectory() as d:
    root = Path(d)
    (root / "README.md").write_text("# Demo\n## Architecture\n## Quick start\n## Results\n
## Limitations\nWhat is simulated: GPUs.")
    (root / "tests").mkdir(); (root / "tests" / "test_gw.py").write_text(&q
uot;def test(): pass")
    (root / "config.py").write_text("API_KEY = 'abcd1234abcd1234abcd'")
    for name, ok in lint(root).items():
        print("PASS" if ok else "FAIL", name)
```

Interview Angle

Q: Pitch your capstone in two minutes. **A:** State the problem (many teams, shared LLM access, isolation and cost control), the design (gateway, virtual keys, per-tenant limits and cache namespaces, aliases), one measured result and one failure you found and fixed. Trade-off: you built a slice, not production, so state what is simulated. Check: each claim maps to a test or RESULTS.md row an interviewer can run.

Q: What would break first if this had 100x the traffic? **A:** Likely the in-process state: counters and buckets need shared storage (Redis) with atomic operations, the cache needs eviction and size budgets, and tracing needs sampling and a durable pipeline. Trade-off: distributed state adds latency and failure modes. Check: a load test with shared state showing added overhead and

behaviour under store failure.

Q: Which claim in your README is least well supported and why? **A:** Answer honestly, for example a latency figure measured on a laptop with the mock model, and say what experiment would firm it up. Trade-off: candour reduces polish but builds trust and shows calibration. Check: the claim is labelled simulated or limited in the README, with the follow-up experiment listed in RESULTS.md.